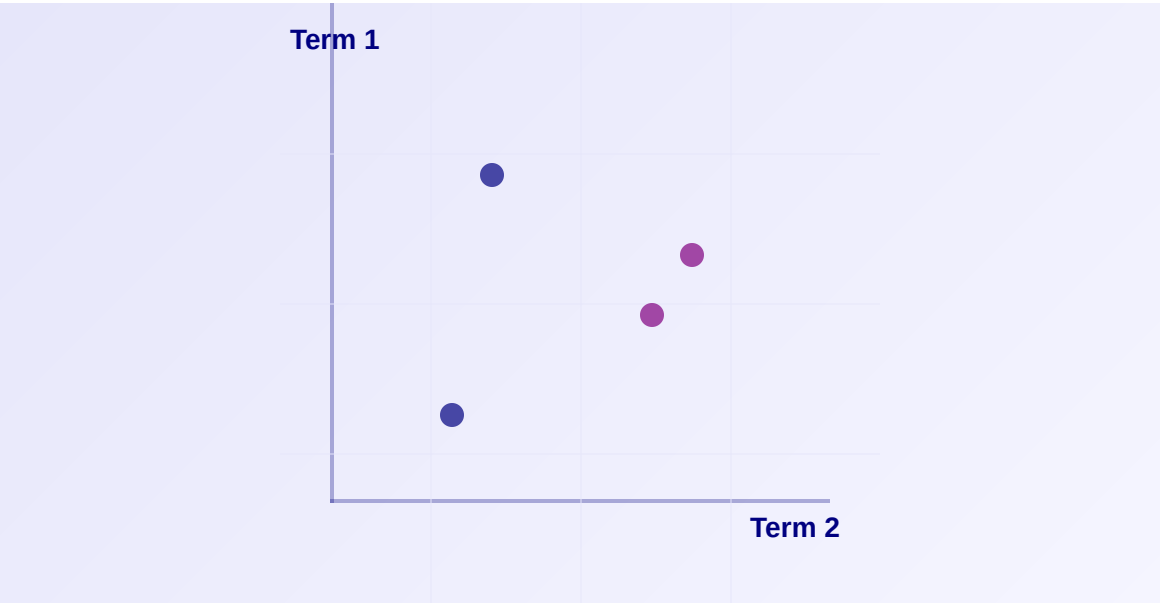
Week 3: Vector Space Model & TF-IDF Scoring

Ranking Documents by Relevance



Course: Information Retrieval

Level: Third Year Computer Science

Duration: 180 minutes (3 hours)

Topics: VSM, TF-IDF, Cosine Similarity, Document Ranking

Week 3 of 12

Course Plan: Information Retrieval (Weeks 1-12)

1. **Introduction to IR**
   Boolean Retrieval, Inverted Indexes
2. **Document Preprocessing**
   Tokenization, Stemming, Normalization
3. **Vector Space Model** — YOU ARE HERE
   TF-IDF, Cosine Similarity
4. **Index Construction**
   Compression, Optimization
5. **Relevance Feedback**
   Query Expansion, Rocchio
6. **Web Crawling**
   Frontier Management, Politeness
7. **PageRank & Link Analysis**
   HITS, Authority, Hub Scores
8. **Evaluation Methodologies**
   Metrics, A/B Testing, User Studies
9. **Clustering & Classification**
   K-means, Naive Bayes, SVM
10. **Learning to Rank**
    Neural IR, RankNet, LambdaMART
11. **Spelling & NER**
    Error Correction, Entity Recognition
12. **Semantic Search**
    Dense Retrieval, Embeddings

Week 3 Learning Objectives

1. Understand the Vector Space Model (VSM) and its advantages over Boolean retrieval
2. Learn TF-IDF term weighting and its mathematical foundations
3. Master cosine similarity for computing document-query relevance
4. Implement efficient document ranking algorithms using VSM
5. Explore modern extensions: LSA, dense vectors, and neural embeddings
6. Apply VSM concepts to real-world search systems and use cases

Limitations of Boolean Retrieval

No Ranking Capability

Boolean retrieval returns a **set** of documents, not a ranked list. All matching documents are considered equally relevant, making it difficult for users to find the "best" result among thousands.

"Feast or Famine"

Queries often return either **too many results** (10,000+) or **zero results**. There is no middle ground to gradually relax constraints without rewriting the query.

No Partial Matching

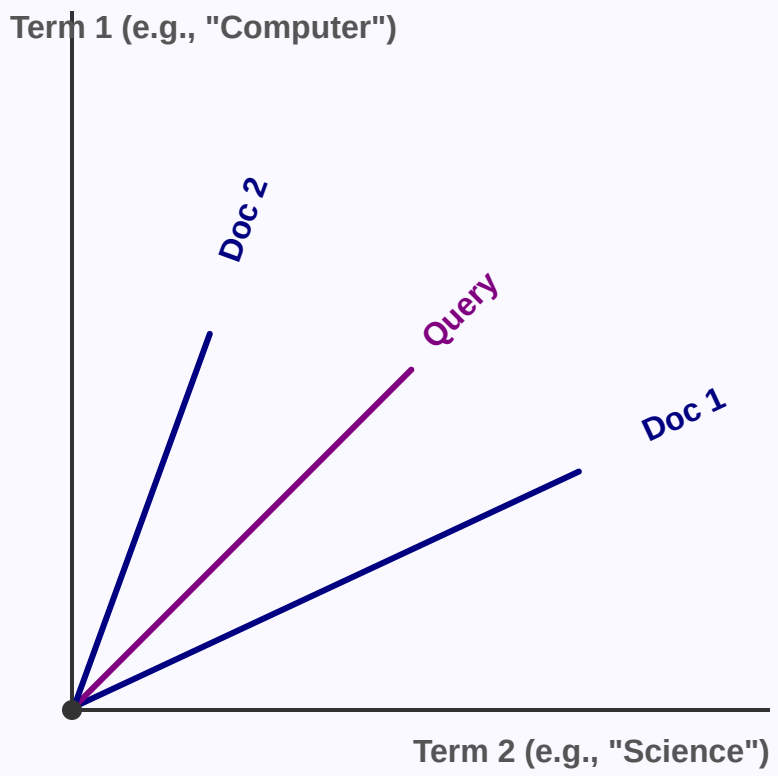
The logic is strict. If a document contains 9 out of 10 query terms but misses one in an `AND` query, it is **completely rejected**, even if it is highly relevant.

Complex Query Formulation

Most users struggle to translate their information needs into complex Boolean logic (e.g., `(A OR B) AND (C OR D) AND NOT E`). Natural language queries are preferred.

Introduction to Vector Space Model

- Documents and queries are represented as **vectors** in a high-dimensional space.
- Each dimension corresponds to a unique **term** in the vocabulary.
- Coordinate values represent the **weight** (importance) of each term.
- Relevance is computed as the **similarity** (distance or angle) between vectors.
- Allows for **partial matching** and ranked results, unlike Boolean retrieval.



Document Vectors

High-Dimensional Representation

Each document is a vector where each dimension is a unique term.

Vector Dimensions

If the vocabulary size is **|V|** (e.g., 100,000), vectors have |V| components.

Term Weights

Each component value shows that term's importance (count or TF-IDF).

Sparsity

Most documents include a small subset of terms, so most components are **zero**.

Term (Dimension)	Weight
algorithm	0.45
bakery	0.00
computer	0.32
cooking	0.00
data	0.58
...	...
zebra	0.00

Example: A "Computer Science" document vector — note zero weights for unrelated terms.

Query Vectors

- **Unified Space:** Queries are treated as "short documents" and represented in the exact same high-dimensional space as the document collection.
- **Vector Construction:** A query vector $\vec{q}$ has dimensions equal to the vocabulary size V.
- **Sparsity:** Since queries are typically short (2-5 words), the vector is extremely sparse (mostly zeros).
- **Weighting Schemes:**
  - **Binary:** 1 if term exists, 0 otherwise.
  - **Weighted:** Can use TF-IDF (though IDF is constant for a single query).

> *"By mapping the query to the same vector space, we transform the retrieval problem into a geometric problem of finding the closest vectors."*

User Query

"machine learning"

Vector Representation $\vec{q}$

Dimension (Term)	Weight ($w_{i,q}$)
algorithm	0
computer	0
learning	1
machine	1
network	0
...	...

Cosine Similarity Metric

- **Measures Orientation:** Calculates the cosine of the angle between two vectors (query and document).
- **Length Invariant:** Normalizes vectors by their magnitude. A short document and a long document with the same term distribution are considered identical.
- **Range [0, 1]:** Since term frequencies are non-negative, the cosine value is always between 0 (orthogonal, no match) and 1 (identical direction).
- **Efficient:** Can be computed efficiently using sparse vector representations (only non-zero dimensions matter).

$$\text{sim}(d, q) = \cos(\theta) = \frac{d \cdot q}{||d|| \times ||q||}$$

Where:

d · q is the dot product (intersection)

||d|| is the magnitude of the document vector

||q|| is the magnitude of the query vector

Why not Euclidean Distance?

Euclidean distance is sensitive to length. A long document would be "far" from a short query even if relevant. Cosine similarity fixes this.

Cosine Similarity Formula

$$\text{sim}(\mathbf{d}, \mathbf{q}) = \frac{\mathbf{d} \cdot \mathbf{q}}{\|\mathbf{d}\| \, \|\mathbf{q}\|} = \frac{\sum (w_{i,d} \times w_{i,q})}{\sqrt{\sum w_{i,d}^2} \times \sqrt{\sum w_{i,q}^2}}$$

× Dot Product (Numerator)

Represents the **intersection** of terms. It is high only if the document and query share high-weighted terms. If they share no terms, it is zero.

Euclidean Norm (Denominator)

Represents the **length** of the vectors. Dividing by the product of lengths normalizes the score, making it independent of document length.

= Result Interpretation

1.0: Vectors point in identical direction (perfect match).
0.0: Vectors are orthogonal (no shared terms).
Intermediate: Degree of semantic overlap.

Cosine Similarity Example

Document (d): [1, 2, 3]
(machine, learning, algorithm)

Query (q): [1, 1, 0]
(machine, learning)

STEP 1: DOT PRODUCT

$$d \cdot q = \sum (d_i \times q_i)$$

$$= (1\times1) + (2\times1) + (3\times0)$$

$$= 1 + 2 + 0$$

3

STEP 2: MAGNITUDES

$$||d|| = \sqrt{(1^2 + 2^2 + 3^2)} = \sqrt{14} \approx 3.74$$

$$||q|| = \sqrt{(1^2 + 1^2 + 0^2)} = \sqrt{2} \approx 1.41$$

3.74 × 1.41 ≈ 5.27

STEP 3: COSINE

$$\cos(\theta) = (d \cdot q) / (||d|| \times ||q||)$$

$$= 3 / 5.27$$

0.57

Result: 0.57 indicates a moderate relevance match.
(1.0 would be a perfect match, 0.0 would be no relevance)

Term Frequency (TF)

The Definition

$tf_{t,d}$ is simply the number of times term *t* appears in document *d*.

Bag of Words Model

We ignore the **order** of words. We only care about the **counts**. "Mary loves John" and "John loves Mary" have the same TF vector.

The Intuition

A document that mentions "query" 10 times is likely more relevant to a search for "query" than a document that mentions it once.

Document d

"The **data** analysis team analyzes the **data** carefully."

Term (t)	Count ($tf_{t,d}$)
data	2
analyzes	1
analysis	1
team	1
carefully	1

TF Weighting Schemes

Raw term frequency is not always the best measure of importance. We often need to dampen the effect of high-frequency terms or normalize for document length.

Scheme	Formula	Description & Use Case
Binary	$1 \text{ if } tf > 0, \text{ else } 0$	Ignores frequency count. Useful when only presence/absence matters (e.g., Boolean retrieval).
Raw Frequency	$tf_{t,d}$	The standard count. Simple, but assumes relevance scales linearly with count (often incorrect).
Log Normalization	$1 + \log(tf_{t,d})$	**Most Common.** Dampens the impact of high frequencies. A term appearing 20 times is not 20x more relevant than appearing once.
Double Normalization	$0.5 + 0.5 \times (tf / max_tf)$	Prevents bias towards longer documents by normalizing against the most frequent term in the document.

Key Insight: The choice of TF variant depends on the specific application. For web search, log normalization is standard because it handles the wide variance in term counts effectively.

Inverse Document Frequency (IDF)

The Problem with Raw Frequency

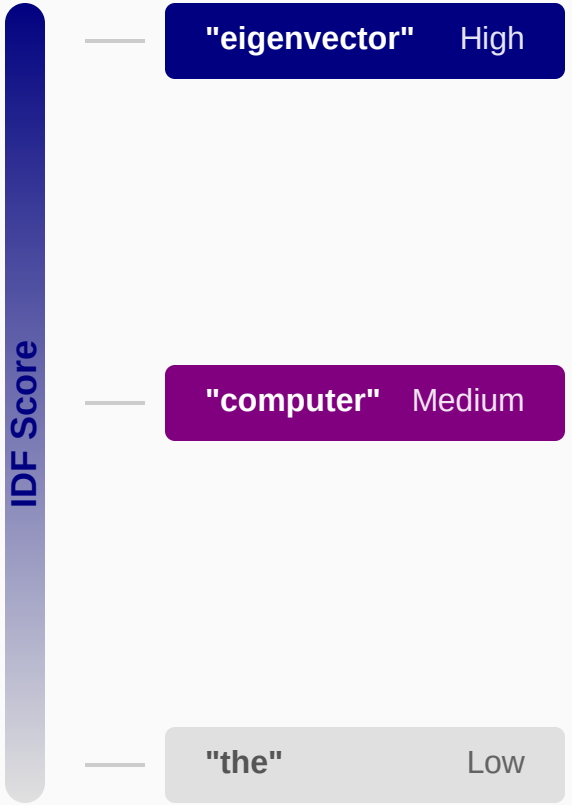
Common words like "the", "is", and "of" appear frequently in almost all documents. Using raw counts gives them disproportionately high influence.

Discriminative Power

Rare terms are more informative. If a user searches for "arachnophobia", that specific term is the key discriminator, not the surrounding common words.

The IDF Solution

We assign a weight that is **inversely proportional** to the number of documents a term appears in. Rare terms get high weights; common terms get low weights.



IDF Formula and Interpretation

$$idf_t = \log_{10}(N / df_t)$$

N Total number of documents

df_t Document frequency of term *t*

Why Logarithm?
It "dampens" the effect. Without log, the weight for rare terms would be astronomically high compared to common terms.

Rare Terms (Low df):
Ratio N/df is large → **High IDF**.

Frequent Terms (High df):
Ratio N/df approaches 1 → log(1) = 0 → **Zero IDF**.

Example: Collection Size N = 1,000,000

Term (t)	df_t	idf_t
calpurnia	1	**6.0**
animal	100	**4.0**
sunday	1,000	**3.0**
fly	10,000	**2.0**
the	1,000,000	**0.0**

TF-IDF Weighting

$$w_{t,d} = tf_{t,d} \times idf_t$$

TF-IDF assigns the highest weights to terms that occur **frequently** in the document but are **rare** in the entire collection.

	Low IDF (Common)	High IDF (Rare)
Low TF	**LOWEST WEIGHT** Rarely mentioned common word (e.g., "the" in a short note)	**LOW WEIGHT** Rare word, but mentioned only once (e.g., "arachnophobia" once)

The TF-IDF Formula

$$w_{t,d} = tf_{t,d} \times idf_t$$

$$= (1 + \log(tf_{t,d})) \times \log(N / df_t)$$

Local Component (TF)

"How important is the term in this specific document?"

Increases with the number of occurrences within the document.

Global Component (IDF)

"How important is the term in the entire collection?"

Increases with the rarity of the term across all documents.

TF-IDF Calculation Example

Doc 1: "the car is green" **Doc 2:** "the car is fast" **Doc 3:** "the bike is blue"

Term: "car" (in Doc 1)

1. TF (Raw Count):	1
2. Document Freq (df):	2 (appears in D1, D2)
3. Total Docs (N):	3
4. IDF ($\log_{10}(N/df)$):	$\log(3/2) \approx 0.176$

Weight = 1 × 0.176 = **0.176**

Term: "the" (in Doc 1)

1. TF (Raw Count):	1
2. Document Freq (df):	3 (appears in all)
3. Total Docs (N):	3
4. IDF ($\log_{10}(N/df)$):	$\log(3/3) = \log(1) = 0$

Weight = 1 × 0 = **0.000**

Key Takeaway: *The common word "the" is completely neutralized (weight 0), while the more specific term "car" retains a positive weight.*

TF-IDF Matrix Representation

The collection is represented as a Term-Document Matrix where each cell contains the computed **weight**, not just the count.

Documents (Columns) →

	Doc 1	Doc 2	Doc 3	...	Doc N
algorithm	**0.45**	0.00	**0.12**	...	0.00
data	**0.33**	**0.58**	0.00	...	**0.21**
mining	0.00	**0.72**	0.00	...	0.00
search	**0.15**	0.00	**0.44**	...	**0.18**
...	...	...	...	...	...

Column Vectors

Each column is a vector \(\vec{d}_j\) representing a document in the vector space.

Sparsity

In real collections (V > 100,000), 99% of cells are **0.00**. We never store the full matrix; we use inverted indexes.

Dimensions

Rows = Vocabulary Size (M)
Columns = Collection Size (N)

Vector Normalization (L2 Norm)

The Problem: Length Bias

Longer documents naturally have higher term frequencies (TF) and more distinct terms. Without normalization, long documents would unfairly dominate search results.

The Solution: Unit Vectors

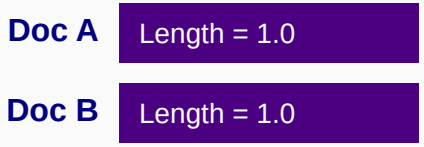
We divide each vector by its length (Euclidean norm). This forces all vectors to have a length of 1.

$$\|d\| = \sqrt{(\sum w_{i,d}^2)}$$

$$w_{norm} = w_{i,d} / \|d\|$$

BEFORE: RAW TF-IDF VECTORS

AFTER: L2 NORMALIZED VECTORS



*Now, only the **direction** (relative term distribution) matters.*

Ranking Documents by Relevance

01 Compute Similarity Scores

For a query *q*, calculate the cosine similarity score for every document *d* in the collection (or a filtered subset).

02 Sort Descending

Order the documents such that:
$score(d_1) \geq score(d_2) \geq ... \geq score(d_N)$

03 Apply Cutoff (Top-K)

Users rarely look beyond the first page. We return only the top *k* results (e.g., k=10 or k=100).

Query: "vector space model"

Introduction to Vector Space Models **0.92**
www.example.edu/ir/vsm-intro
The **vector space model** is an algebraic model for representing text documents...

TF-IDF Weighting Explained **0.78**
www.datascience.com/tfidf
Learn how term frequency and inverse document frequency work in the **vector model**...

Linear Algebra for Search **0.45**
www.math-basics.org/linear-alg
Understanding dot products and cosine similarity is essential for modern search...

Vector Space Retrieval Algorithm

```
CosineScore(query q):
  float Scores[N] = 0
  float Length[N] // Pre-computed doc lengths

  for each term t in q:
    w_tq = CalculateWeight(t, q)
    postings = GetPostingsList(t)

    for each pair (d, tf_td) in postings:
      Scores[d] += w_tq * w_td(tf_td)

  for each d in Scores > 0:
    Scores[d] = Scores[d] / Length[d]

  return TopK(Scores)
```

1 Accumulation Strategy

Instead of iterating over documents (slow), we iterate over **query terms**. We only touch documents that contain at least one query term.

2 Pre-computed Lengths

Document vector lengths (||d||) are static. We compute and store them at indexing time to save CPU cycles during retrieval.

3 Sparse Traversal

The algorithm efficiently handles sparsity. If a term isn't in the query, it contributes 0, so we simply skip it.

Efficiency Note: This approach is much faster than a full matrix multiplication because the query vector is extremely sparse.

Computational Efficiency

The Bottleneck: A naive implementation computes the cosine score for *every* document in the collection. For 10 billion web pages, this is impossibly slow ($O(N)$).

Inverted Indexing

We only process documents that contain at least one query term. This reduces the search space from billions to thousands ($O(L_q)$).

Index Pruning

We can ignore terms with very low weights (stop words) or documents with very low term frequencies during index construction.

Early Termination

We stop processing once we have found the likely top-k results. We don't need the exact score for the 1,000,000th document.

Sparse Vector Representation

LOGICAL VIEW (MENTAL MODEL)

0	0.4	0	0
0	0	0	0.9
0.2	0	0	0
0	0	0.5	0
0	0	0	0

Storing all zeros is wasteful.
(99% of space wasted)

PHYSICAL VIEW (STORAGE)

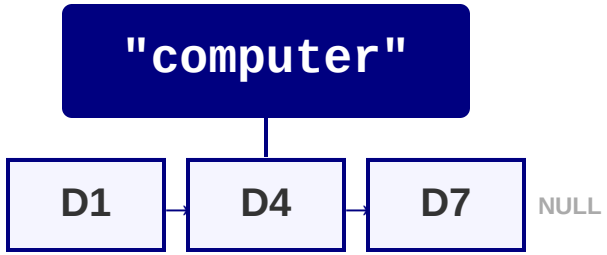
```
Doc 1: [(term2, 0.4)]
Doc 2: [(term4, 0.9)]
Doc 3: [(term1, 0.2)]
Doc 4: [(term3, 0.5)]
Doc 5: []
```

1,000,000	~150	99.98%
Vocabulary Size	Avg Terms per Doc	Sparsity

Inverted Indexes in VSM

To support ranked retrieval, the inverted index must store more than just document presence. It must store the **frequency information** needed to compute weights.

BOOLEAN INDEX

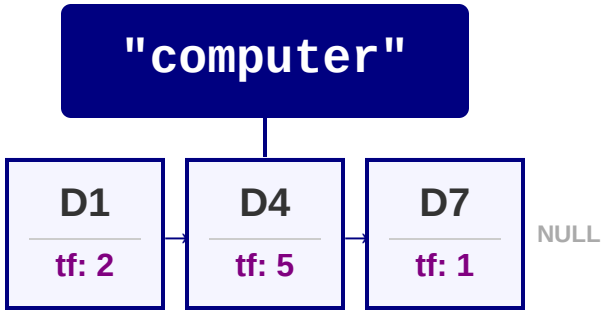


Stores: Doc IDs only

Question: "Does D1 contain 'computer'?"

Result: Yes/No

VSM INDEX



Stores: Doc IDs + **Term Frequency**

Question: "How relevant is D1 for 'computer'?"

Result: Score Contribution

Early Termination Strategies

For common terms, postings lists can contain millions of documents. We need heuristics to stop processing once we likely have the top-k results.

Champion Lists

Static Pruning: At indexing time, we pre-compute the top *r* documents (e.g., 50) with the highest weights for each term.

At query time, we search these "Champion Lists" first.

```
Term: "news"
High-Tier: [D5, D9, D12]
Low-Tier: [D1, D2, D3...]
```

Impact Sorting

Dynamic Pruning: Store postings sorted by weight ($w_{t,d}$) descending, not by DocID.

Process high-impact documents first and stop when remaining possible score cannot beat the k-th best.

```
Term: "news"
Postings: (D9: 0.9) → (D5: 0.8) → (D12: 0.2)...
```

The Trade-off: These are "unsafe" optimizations. We gain massive speed (10x-100x) but might occasionally miss a relevant document that relies on many small weights.

Query Expansion in VSM

The Vocabulary Mismatch

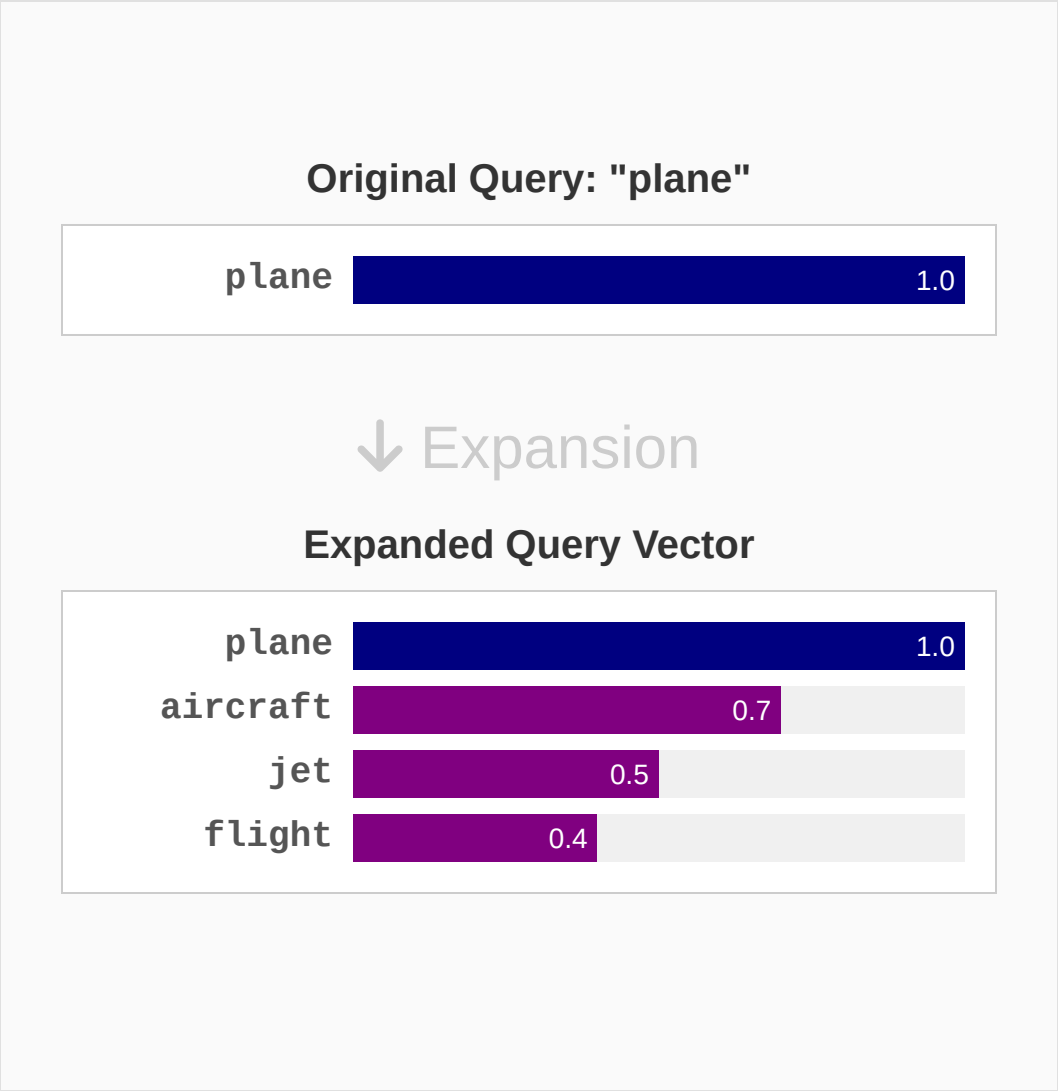
Users query for "plane", but relevant documents might use "aircraft" or "jet". Exact matching fails here.

Vector Augmentation

We mathematically "stretch" the query vector into new dimensions corresponding to synonyms or related terms.

Reweighting Strategy

Original terms keep high weights (e.g., 1.0). Expanded terms get **lower weights** (e.g., 0.5) to reflect lower confidence.



Pseudo-Relevance Feedback

> Also known as **"Blind Relevance Feedback"**. It automates the feedback loop by assuming the system's initial top results are relevant, without requiring any user interaction.

1. Initial Search

Run the user's original query using standard retrieval (e.g., TF-IDF).

→

2. Assumption

Assume the top *k* (e.g., 10) returned documents are relevant.

→

3. Expansion

Extract frequent terms from these *k* docs and add them to the query.

→

4. Final Search

Run the new, expanded query to get the final result list.

The Risk: Query Drift

If the initial top *k* documents are actually irrelevant (e.g., due to polysemy), the algorithm will add bad terms (noise) to the query, steering the results further away from the user's intent.

Advantages of Vector Space Model

Ranked Retrieval

Unlike Boolean retrieval which returns an unordered set, VSM sorts documents by **relevance score**. Users see the best results first.

Partial Matching

A document can be retrieved even if it doesn't contain *all* query terms. This solves the "zero results" problem of strict Boolean AND queries.

Term Weighting

Through TF-IDF, VSM automatically recognizes that matching unique terms (e.g., "arachnophobia") is more valuable than matching common terms (e.g., "fear").

Query Simplicity

Users can type natural language (free text) without needing to learn complex Boolean operators or query syntax.

> *"VSM transforms information retrieval from a binary decision (match/no-match) into a continuous ranking problem."*

Limitations of Vector Space Model

Term Independence

VSM assumes terms are orthogonal (independent) dimensions. It ignores relationships between words.

```
"New York" is treated as "New" + "York", losing
the specific geographic meaning.
```

Semantic Gap

It matches exact keywords, not meanings. It fails to handle **Synonymy** (different words, same meaning) without expansion.

```
Query "automobile" has 0 similarity with a
document containing only "car".
```

Loss of Structure

The "Bag of Words" model discards word order and syntactic structure, leading to loss of meaning.

```
"Mary loves John" is identical to "John loves
Mary" in VSM.
```

Lack of Context

It fails to handle **Polysemy** (same word, different meanings) because it aggregates all occurrences into one vector dimension.

```
"Apple" (fruit) and "Apple" (tech company) are
conflated.
```

The Term Independence Assumption

"Bag of Words" Model: VSM assumes that terms occur independently of one another. It treats words as orthogonal dimensions, completely ignoring grammar, word order, and proximity.

LOSS OF SYNTAX

"Mary loves John"

=

"John loves Mary"

`Vector: {Mary:1, loves:1, John:1}`

VSM cannot distinguish subject from object.

LOSS OF SEMANTICS

"White House" (Politics)

=

"House painted White" (Real Estate)

`Vector: {White:1, House:1}`

VSM breaks the phrase into independent concepts.

Mathematical Assumption: $P(t_1, t_2) = P(t_1) \times P(t_2)$
(This is almost never true in natural language!)

Polysemy & Synonymy Issues

The Vector Space Model relies on exact term matching. It fails to capture the **semantic meaning** behind the words, leading to two major problems.

POLYSEMY

"One word, multiple meanings"

`"Jaguar"` ↔ (Car) (Animal)

Impact: Low Precision
Retrieves irrelevant documents (noise).

SYNONYMY

"Multiple words, same meaning"

`"Movie"`
`"Film"`
→ (Cinema)

Impact: Low Recall
Misses relevant documents that use different words.

Real-World Example: News Search

USER QUERY

"economic crisis inflation"

1. System tokenizes query and builds vector **q**.
2. "Crisis" and "Inflation" have high IDF (rare/important).
3. System computes Cosine Similarity with millions of articles.

Central Banks Struggle with Rising Inflation

...the global economic outlook worsens as the crisis deepens. Inflation rates have hit a 40-year high...

`TF(inflation)=5, TF(crisis)=3`

Score: 0.92

Tech Stocks Rally on Economic News

...positive economic data boosted the market today. Investors ignored the potential risks...

`TF(economic)=1, TF(inflation)=0`

Score: 0.35

Real-World Example: E-commerce Search

Fielded Retrieval

Products are structured documents (Title, Brand, Description, Category). VSM is applied to each field independently.

Weighted Aggregation

Matches in the **Title** or **Brand** are far more significant than matches in the long Description. We assign weights (α, β, γ) to boost specific fields.

$$\text{Score} = 3.0 \times \text{Sim}(\text{Title}) + 2.0 \times \text{Sim}(\text{Brand}) + 1.0 \times \text{Sim}(\text{Desc})$$

Query: " **sony noise cancelling** "

BRAND **Sony** Weight: 2.0

TITLE WH-1000XM5 Wireless **Noise Cancelling** Headphones Weight: 3.0

DESC Industry-leading **Sony** processors... perfect for **noise cancelling** environments. Weight: 1.0

The product ranks highly because the query terms hit the high-weight fields.

Real-World Example: Academic Search

The Challenge

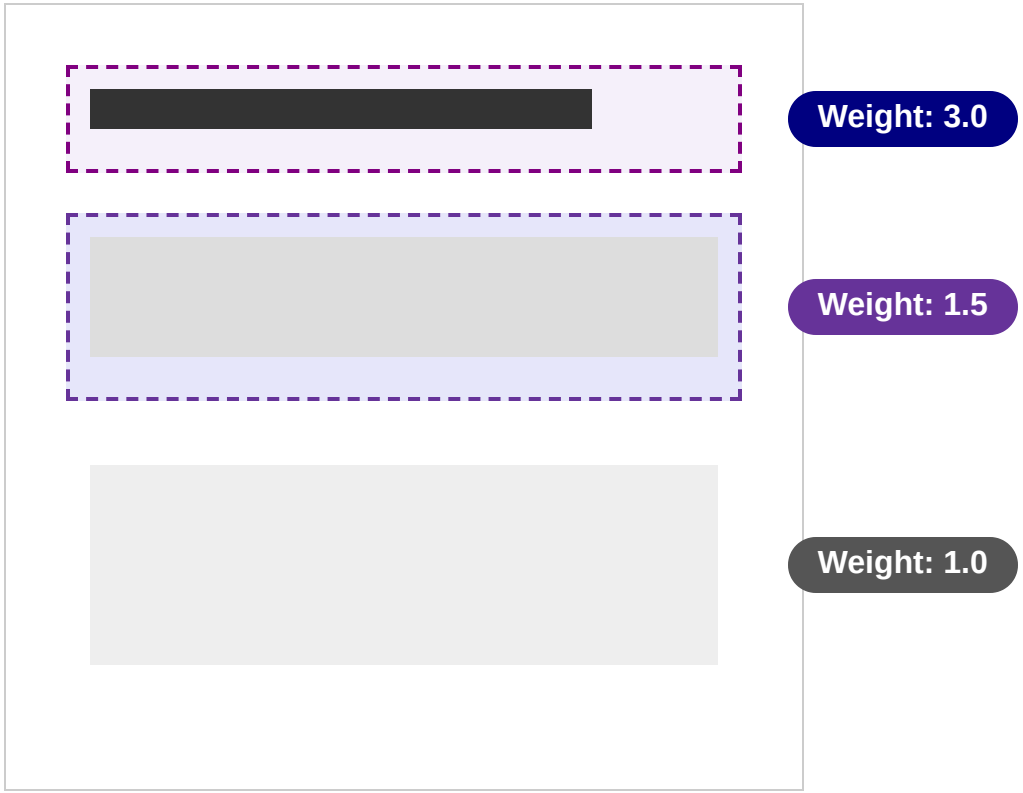
Users often search with long queries (e.g., entire problem statements) or expect results that match specific technical jargon.

Fielded VSM

A document isn't just one bag of words. It has structure. We calculate separate cosine similarities for each field and compute a **weighted sum**.

"More Like This"

The system uses the entire vector of a relevant paper as the query vector to find nearest neighbors in the high-dimensional space.



$$\text{Score} = 3.0 \times \text{Sim}(q, \text{Title}) + 1.5 \times \text{Sim}(q, \text{Abstract})$$

TF-IDF Variants (SMART Notation)

Standard notation **ddd.qqq** (Document.Query) describes the weighting scheme used.

1. Term Frequency

- **n** — **Natural**
  $tf_{t,d}$
- **l** — **Logarithm**
  $1 + \log(tf_{t,d})$
- **a** — **Augmented**
  $0.5 + 0.5(tf / max_tf)$

2. Doc Frequency

- **n** — **None**
  1
- **t** — **IDF**
  $\log(N / df_t)$
- **p** — **Probabilistic**
  $\log((N - df) / df)$

3. Normalization

- **n** — **None**
  1
- **c** — **Cosine**
  $1 / \sqrt{(w_1^2 + \ldots + w_M^2)}$
- **u** — **Pivoted Unique**
  $1 / u$

Example: `lnc.ltc`

Doc: Log TF, No IDF, Cosine Norm
Query: Log TF, IDF, Cosine Norm

Beyond TF-IDF: Okapi BM25

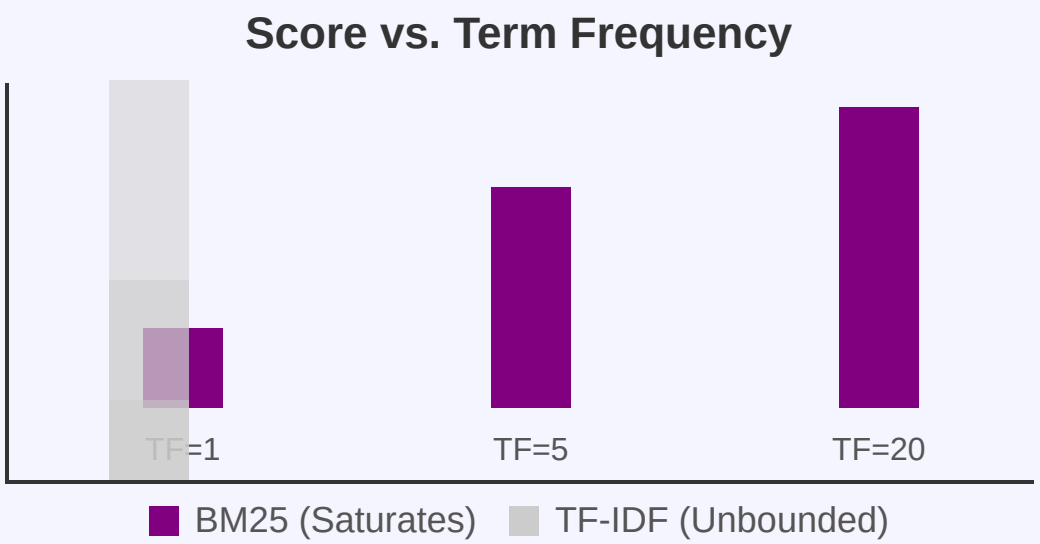
BM25 is the industry-standard probabilistic retrieval function that fixes two key TF-IDF issues.

TF Saturation

In TF-IDF, score can grow without bound. BM25 makes the score saturate so extra repeats have diminishing impact.

Length Normalization

BM25 penalizes overly long (verbose) documents via a tunable parameter **b**, while allowing comprehensive docs less penalty.



$$\text{Score} \propto ((k_1 + 1) * tf) / (k_1 * (1 - b + b * L/L_{avg}) + tf)$$

Language Models for IR

Instead of measuring geometric distance (VSM), we ask a probabilistic question: **"What is the probability that this document generated this query?"**

Document (d) → **Language Model (M_d)** → **Query (q)**

$$P(q \mid M_d) = \prod P(t \mid M_d)$$

VECTOR SPACE MODEL

"How similar are vectors **d** and **q**?"
(Geometric Angle)

LANGUAGE MODEL

"How likely is **d** to speak **q**?"
(Generative Probability)

Geometric Interpretation

The Term Space

Imagine a high-dimensional space where each axis represents a unique term in the vocabulary. A vocabulary of 100,000 words creates a 100,000-dimensional space.

Direction = Meaning

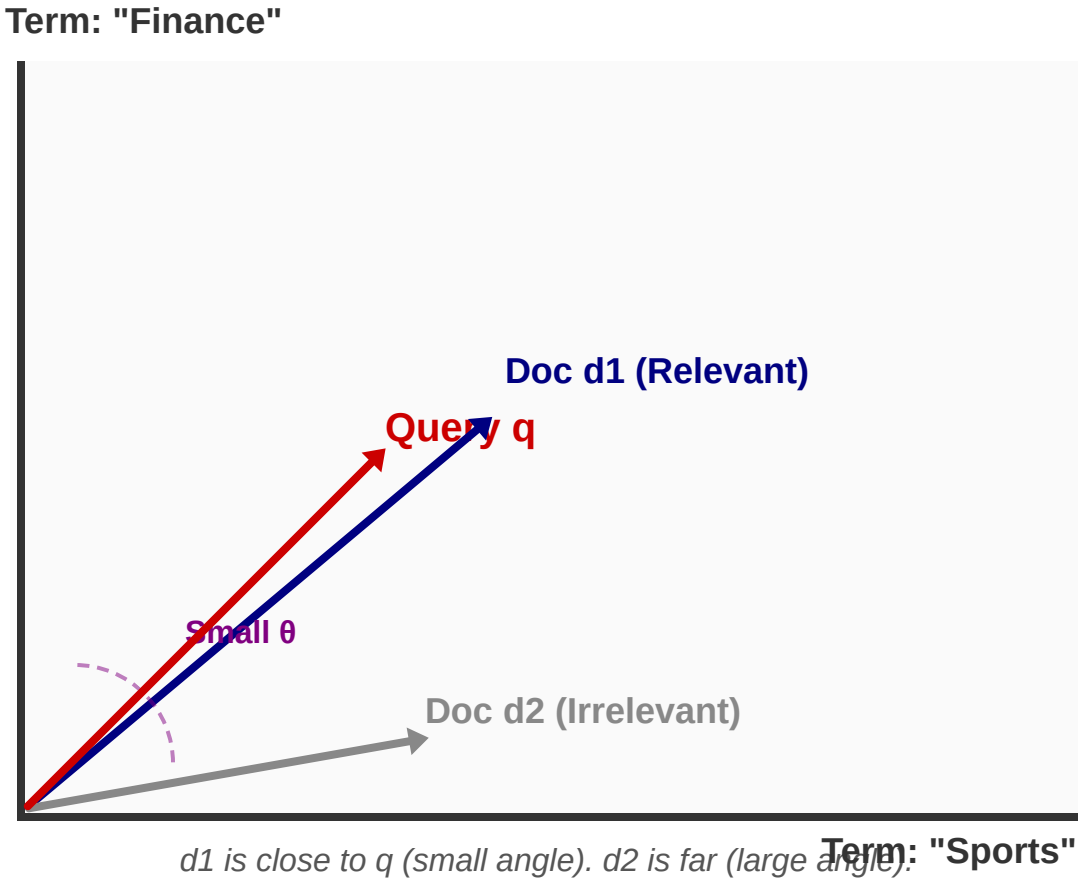
The **direction** of a vector represents the semantic content (the ratio of terms). Documents about "cars" point in one direction; documents about "cooking" point in another.

Angle = Similarity

The **cosine of the angle** between two vectors measures their similarity.

$\theta = 0° \rightarrow$ Cosine = 1 (Identical)

$\theta = 90° \rightarrow$ Cosine = 0 (Unrelated)



d1 is close to q (small angle). d2 is far (large angle).

High-Dimensional Spaces

In Information Retrieval, the "space" is defined by the vocabulary. Every unique word adds a new dimension (axis) to the geometry.

2D Plane

2 Axes

3D Space

3 Axes

IR Vector Space

100,000+ Axes

The Empty Void

The space is incredibly vast and mostly empty. Documents are tiny points floating in a massive void, rarely touching each other.

Orthogonality

If you pick two random vectors in a high-dimensional space, they are almost certainly **orthogonal** (90° angle, 0 similarity).

Distance

Euclidean distance becomes less meaningful because all points tend to be equidistant from the center. **Cosine similarity** remains robust.

The Curse of Dimensionality

Exponential Volume

As dimensions increase, the volume of the space grows exponentially. To maintain the same data density, the number of required data points must also grow exponentially.

Extreme Sparsity

In a 100,000-dimensional space, even a billion documents are like a few grains of sand scattered across a galaxy. Most of the space is empty.

Distance Concentration

In high dimensions, the ratio of the distance to the nearest neighbor vs. the farthest neighbor approaches 1.

"Everyone is equally far away."

$$\lim_{d \to \infty} (Dist_{max} - Dist_{min}) / Dist_{min} \to 0$$

Algorithmic Failure

Standard spatial indexing structures (like k-d trees or R-trees) become ineffective. They degrade to linear scan performance (O(N)).

Dimensionality Reduction Techniques

To combat the Curse of Dimensionality and improve efficiency, we reduce the number of features (terms) using two primary strategies.

FEATURE SELECTION

"Keep the best, drop the rest."

- Select a subset of original terms.
- **Methods:** Frequency pruning, Mutual Information, Chi-Square test.
- **Pros:** Preserves original meaning.

```
Input: {car, auto, the, a}
Output: {car, auto}
```

FEATURE EXTRACTION

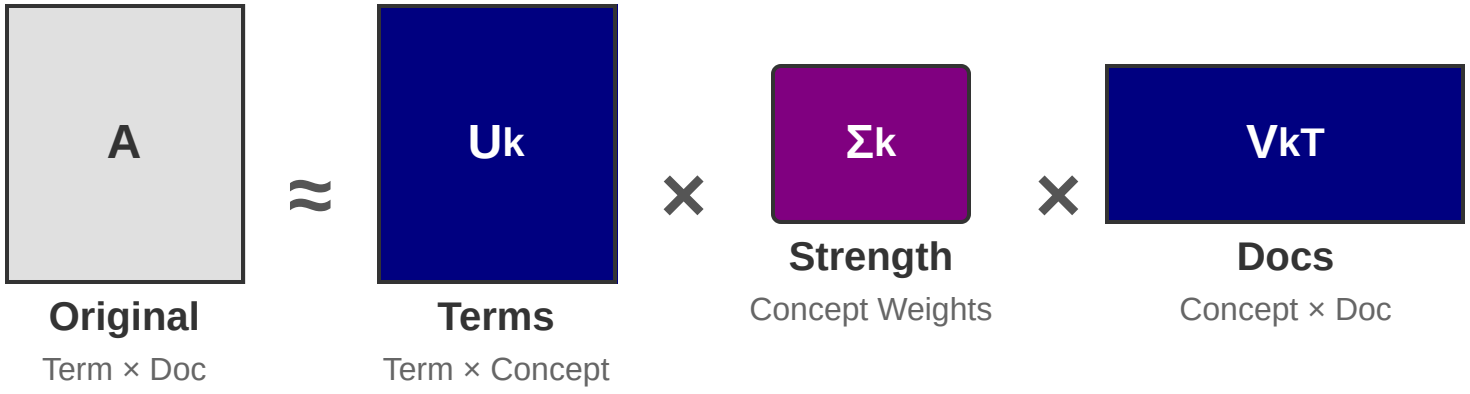
"Create new, better dimensions."

- Project data into a new lower-dimensional space.
- **Methods:** LSA (SVD), PCA, Autoencoders.
- **Pros:** Handles synonyms/polysemy.

```
Input: {car, auto, vehicle}
Output: {Concept_1 (Transport)}
```

Latent Semantic Analysis (LSA)

LSA uses **Singular Value Decomposition (SVD)** to decompose the term-document matrix into three component matrices, revealing hidden (latent) semantic structures.



Dimensionality Reduction

By keeping only the top *k* singular values (in Σ), we force the system to ignore noise and merge similar terms. **"Car"** and **"Automobile"** get mapped to the same latent concept dimension.

Benefits of LSA

By projecting data into a lower-dimensional semantic space, LSA overcomes the two fundamental limitations of keyword matching.

Handles Synonymy

LSA recognizes that different words occurring in similar contexts refer to the same underlying concept.

```
"Car" & "Auto"
↓
Mapped to Concept_1
```

Result: Higher Recall

Handles Polysemy

LSA uses the global context of the document to disambiguate words with multiple meanings.

```
"Bank" (River) vs "Bank" (Money)
↓
Separated into Concept_2 & Concept_3
```

Result: Higher Precision

Noise Reduction: By discarding the smallest singular values (the "tail" of the spectrum), LSA filters out random noise, typos, and idiosyncratic word usage, leaving only the strong semantic signals.

Implementation Considerations

Implementing VSM at scale requires addressing storage efficiency, computational speed, and maintenance costs.

Sparse Storage

The term-document matrix is 99.9% zeros. **Never** store the full matrix. Use inverted lists to store only non-zero entries (DocID, Weight).

```
Map<Term,
List<Posting>>
index;
```

Pre-computed Norms

Computing Euclidean length $|d|$ at query time is too slow. Compute and store $1/|d|$ for every document during indexing.

```
float[] docNorms
= new float[N];
```

Dynamic IDF

IDF values change as new documents are added. Re-computing global stats is expensive. Use periodic updates or approximations.

```
update_stats(interval='daily');
```

Numeric Precision

Storing 64-bit doubles for weights is wasteful. Use 32-bit floats or even **quantized integers** (1-byte) to save RAM.

```
int8_t weight =
quantize(float_val);
```

Data Structures for VSM

The Term-Document matrix is 99.9% sparse. Storing it as a 2D array is impossible. We use specialized structures to store only non-zero values.

INVERTED INDEX

Maps **Terms** → **Documents**.
Crucial for: Fast query processing.
VSM Modification: Store TF or weights in the postings payload.

```
"apple" → [D1: 2] → [D5: 1]
"pie"   → [D2: 4] → [D5: 3]
```

FORWARD INDEX

Maps **Documents** → **Terms**.
Crucial for: Snippet generation, Re-ranking, and computing document length (L_d).

```
Doc D1 → "apple", "fruit", "red"...
Doc D2 → "pie", "sugar", "bake"...
```

Efficiency Tip: Modern systems often compress these lists using techniques like *Variable Byte Encoding* or *PForDelta* to save RAM and disk I/O.

Indexing Strategies

Building an index for massive collections requires specialized algorithms to manage memory constraints and disk I/O efficiency.

BSBI

BLOCK SORT-BASED

- Accumulate (TermID, DocID) pairs in RAM.
- **Sort** pairs by TermID.
- Write sorted block to disk.
- Merge all blocks into final index.

Pros: Simple logic.
Cons: Sorting is expensive.

SPIMI

SINGLE-PASS IN-MEMORY

- Generate dictionary dynamically.
- Add postings directly to lists.
- **No sorting** of pairs required.
- Write to disk when RAM is full.

Pros: Faster (O(N)).
Cons: Complex memory mgmt.

MapReduce

DISTRIBUTED INDEXING

- **Map:** Parse docs, emit (Term, DocID).
- **Shuffle:** Group by Term.
- **Reduce:** Create postings lists.
- Scale to petabytes of data.

Pros: Infinite scale.
Cons: Network overhead.

Query Processing in VSM

In VSM, the query is treated as a "short document." The system must convert user intent into a vector representation compatible with the index.

User Input
"cheap apple laptop"

→

Preprocessing
Tokenize, Stem, Stopwords

→

Vectorize
Compute TF-IDF weights

→

Scoring
Cosine Similarity

Phrase Queries

Problem: VSM is "bag-of-words" (ignores order).

Solution: Use VSM for initial retrieval, then apply a **post-retrieval filter** to check positional constraints (slow) or use a phrase index.

Boolean Filters

Problem: "Must contain X" is a hard constraint.

Solution: **Filter first** using Boolean logic to get a subset of documents, then rank that subset using VSM scoring.

Wildcards

Problem: "compu" isn't in the dictionary.*

Solution: Expand "compu*" to {computer, computing, compute}, then create a **weighted union** query vector.

Performance Metrics for VSM

Unlike Boolean retrieval (which returns a set), VSM returns a **ranked list**. We must evaluate the quality of the ranking, not just the presence of documents.

Set-Based Metrics

Precision & Recall

Standard metrics for unranked sets.
Precision: % of retrieved that are relevant.
Recall: % of relevant that are retrieved.

$$F1 = 2 * (P * R) / (P + R)$$

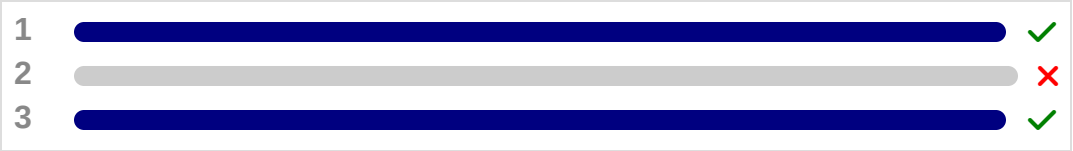
Limitation

These metrics treat the document at Rank #1 the same as the document at Rank #1000. They ignore user attention span.

Rank-Aware Metrics

Precision@k (P@k)

Precision calculated only for the top *k* results (e.g., P@10). Crucial for web search where users rarely look past page 1.



MAP & NDCG

MAP: Mean Average Precision (for binary relevance).
NDCG: Normalized Discounted Cumulative Gain (for graded relevance, e.g., 5 stars > 3 stars).

$$NDCG_p = DCG_p / IDCG_p$$

Case Study: Elasticsearch

Elasticsearch is a distributed, RESTful search and analytics engine built on Apache Lucene. It is the industry standard for implementing VSM/BM25 at scale.

Distributed Architecture

Data is split into **shards** and distributed across nodes. Scoring is computed per shard and aggregated (Query-Then-Fetch).

JSON over HTTP

Queries are defined in a flexible JSON Domain Specific Language (DSL). No SQL required.

Relevance Scoring

Uses **BM25** (an advanced probabilistic version of TF-IDF) as the default similarity algorithm since version 5.0.

```
GET /products/_search

"query": { "match": { "description": "wireless headphones" } }, // Response (Simplified) "hits": [ { "_index": "products", "_id": "42", "_score": 2.85, // The VSM/BM25 Score "_source": { "name": "Sony Noise Cancelling..." } } ]
```

Note: The `_score` is a floating-point number representing how well the document matches the query. Higher is better. It is not a percentage.

Case Study: Apache Solr

Built on Apache Lucene, Solr provides a robust platform for applying VSM and Probabilistic models to enterprise data.

Pluggable Similarity

Allows switching between TF-IDF, BM25, or custom DFR models at runtime via `schema.xml`.

eDisMax Parser

Extended Disjunction Max. Maps user queries to multiple fields with different boost weights (e.g., Title^10).

Function Queries

Modify the VSM score using mathematical functions on numeric fields (e.g., `log(popularity)`).

```
// Example: Boost titles and recent documents
q="iphone case"
defType=edismax
qf=title^10 description^2 category^1  // Query Fields & Weights
bf=recip(ms(NOW,date_field),3.16e-11,1,1) // Recency Boost
```

Modern: Neural Vector Representations

Traditional VSM counts words. Modern Neural IR understands meaning by mapping text to **dense vectors** (embeddings) where semantic relationships are preserved geometrically.

SPARSE (VSM/TF-IDF)

```
[0, 0, 1, 0, 0, 0, 5, 0...]
```

- High Dimensional (100k+)
- Mostly Zeros
- Exact Keyword Match

DENSE (NEURAL)

```
[0.12, -0.54, 0.88, 0.03...]
```

- Low Dimensional (768)
- All Non-Zero Floats
- Semantic Match

Vector Arithmetic: *vec("King") - vec("Man") + vec("Woman") ≈ vec("Queen")*

Word Embeddings

Moving from sparse, high-dimensional vectors (TF-IDF) to dense, low-dimensional semantic vectors learned from massive text corpora.

King - Man + Woman ≈ Queen

Word2Vec

PREDICTIVE MODEL (GOOGLE, 2013)

Uses a shallow neural network to predict words from their context.

- **CBOW:** Predict target word from surrounding context.
- **Skip-gram:** Predict context words from target word.
- Excels at capturing local context windows.

GloVe

COUNT-BASED MODEL (STANFORD, 2014)

Global **Ve**ctors. Factorizes the logarithm of the global word co-occurrence matrix.

- Combines matrix factorization with local context window.
- Explicitly encodes global statistics.
- Often faster to train on smaller corpora.

"You shall know a word by the company it keeps." — J.R. Firth (1957)

Document Embeddings

To retrieve documents, we need to aggregate individual word vectors into a single, fixed-length vector representing the entire text.

Centroid (Average)

Simply take the arithmetic mean of all word vectors in the document.

$$v_d = (1/N) \sum v_w$$

+ Fast, surprisingly effective baseline.
− Ignores word order ("Dog bites man" = "Man bites dog").

Doc2Vec (PV-DM)

Extends Word2Vec by adding a unique "Paragraph ID" vector that acts as a memory for the context.

$$P(w_t \mid w_{t-k} \ldots w_{t+k}, D)$$

+ Learns topic & style.
− Requires inference step for new docs.

Transformers (BERT)

Uses self-attention to generate context-aware embeddings. Uses the `[CLS]` token or mean pooling.

$$v_d = \mathrm{BERT}(\mathrm{doc})_{[CLS]}$$

+ State-of-the-art accuracy.
− Computationally expensive (GPU required).

Sparse vs. Dense Vectors

Choosing the right representation involves a trade-off between exactness and understanding. Modern systems often use both.

FEATURE	SPARSE (TF-IDF / BM25)	DENSE (EMBEDDINGS)
Matching	Exact Keyword Match	Semantic Meaning
Dimensions	High (100,000+)	Low (128 - 1024)
Interpretability	High (Readable terms)	Low (Black box numbers)
Vocabulary	Fails on synonyms / OOV	Handles synonyms well
Best For	Part numbers, Names, Code	Concepts, Questions, Intent

The Solution: Hybrid Search

Combine scores from both models: `Score = α * BM25 + (1 - α) * Cosine(Dense)`. This gives the best of both worlds.

Future Directions

The future of Information Retrieval lies in the convergence of exact keyword matching, semantic understanding, and extreme-scale engineering.

Hybrid Search

Combining the precision of Sparse Vectors (BM25) with the recall of Dense Vectors (Embeddings).
Technique: **Reciprocal Rank Fusion (RRF)**.

Multimodal Retrieval

Searching across data types (Text-to-Image, Image-to-Video). Models like **CLIP** map images and text into the same vector space.

Learned Sparse (SPLADE)

Using Transformers to generate sparse vectors and expand queries with relevant terms.
Benefit: **BERT-level quality** with Inverted Index speed.

Hardware Acceleration

Indexing billion-scale vectors using GPU acceleration and ANN graphs like **HNSW**.

Summary: Key Takeaways

Text is Geometry

The Vector Space Model transforms text retrieval into a geometric problem. Documents are points in a high-dimensional space; **similarity is proximity**.

TF-IDF Weighting

Raw frequency counts are insufficient. We must balance local frequency (TF) with global rarity (IDF) to identify truly significant terms.

Cosine Similarity

The angle between vectors is a robust measure of similarity that naturally handles documents of varying lengths, unlike Euclidean distance.

Efficient Implementation

Real-world VSM requires sparse matrix storage (Inverted Indexes), pre-computed norms, and optimized scoring algorithms to scale.

---

*"Ranking is not just about finding matches; it's about finding the *best* matches."*

Next Week: Index Construction & Compression

We know *what* an index is. Next week, we shift from theory to engineering: **How do we build it efficiently at web scale?**

Hardware Constraints

Understanding the massive speed gap between RAM and Disk I/O. Why random access kills performance.

Scalable Algorithms

BSBI (Block Sort-Based Indexing) and **SPIMI** (Single-Pass In-Memory Indexing) for handling collections larger than RAM.

Distributed Indexing

Using **MapReduce** to parallelize index construction across thousands of machines.

Index Compression

Techniques like **Variable Byte** and **Gamma Codes** to shrink the index by 50%+.

Key Insight: Compression isn't just about saving disk space; it's about caching more in memory for speed.

Questions & Discussion

DISCUSSION PROMPTS:

- How does the Vector Space Model handle synonyms (e.g., "car" vs. "automobile")?
- Why is Cosine Similarity preferred over Euclidean Distance for text documents?
- What are the computational challenges of calculating cosine scores for millions of documents?

Thank you for your attention.